



PENSAMIENTO ALGORÍTMICO
SECUENCIAS,
CONDICIONES Y CICLOS

SECUENCIAS, CONDICIONES Y CICLOS

La lógica algorítmica hace referencia a su estructura y orden, sus componentes claves son:

- Secuencias: ejecución de instrucciones de manera ordenada y jerárquica, lineal, una tras otra y sin interrupción, hasta su finalización.

Ejemplo

Preparar un café implica:



1. Hervir agua.
2. Moler café.
3. Poner el café molido en el filtro.
4. Verter el agua caliente sobre el café.
5. Esperar el filtrado del café.
6. Servir en una taza.

La ejecución en cada línea de la secuencia, está definida de manera tal que permita organizar lógicamente el paso a paso y asegurar el éxito en la ejecución del algoritmo.

- Condiciones: al momento de requerirse la utilización de la lógica, se implementan las condiciones que permiten al algoritmo tomar decisiones y continuar. Las expresiones lógicas devuelven el valor verdadero o falso y se utilizan estructuras condicionales como el if, else, en la mayoría de los lenguajes de programación.

- if: se realiza la operación si se cumple con la condición.
- else: se evalúa otra condición si la principal no se cumple.

- Condiciones: al momento de requerirse la utilización de la lógica, se implementan las condiciones que permiten al algoritmo tomar decisiones y continuar. Las expresiones lógicas devuelven el valor verdadero o falso y se utilizan estructuras condicionales como el if, else, en la mayoría de los lenguajes de programación.

Tabla 1
Ejemplo tipo de condición

Tipo de condición	Ejemplo
<i>if</i>	<i>If (Edad>18) then</i> "eres mayor de edad"
<i>else</i>	<i>If (Edad>18) then</i> "eres mayor de edad" <i>else</i> "eres menor de edad"

Existen dos tipos de condiciones: **simples**, cuando se evalúa una sola condición y **múltiples**, si se hace necesario la utilización de varias condiciones.

Las condiciones permiten adaptar al algoritmo a diferentes situaciones, tomando decisiones, según la disponibilidad de los datos recibidos y el procedimiento a ejecutar.

- Ciclos: conjunto de instrucciones de repetición que se llevan a cabo mientras se cumple una condición. Su importancia radica en la posibilidad de reducir la redundancia en el código y automatizar secuencias repetitivas.

Los tipos de ciclo son:

while: mientras. Se lleva a cabo la instrucción y se repite mientras la condición sea verdadera.

Tabla 2
Ciclo *while*

while	Explicación
<pre>cont = 0; while (cont <= 5) cont = cont + 1;</pre>	Se inicializa el valor de cont en 0. Mientras cont <= 5, se ejecuta el código cont incrementa su valor en 1. repite

La ejecución en cada línea de la secuencia, está definida de manera tal que permita organizar lógicamente el paso a paso y asegurar el éxito en la ejecución del algoritmo. Para el algoritmo anterior, se realizará la ejecución del código 5 veces puesto que cont se incrementa de uno en uno.

for: hasta. Se reproduce un total finito de veces, es decir; se conoce el inicio y el fin del ciclo.

Su estructura está definida en inicialización, fin, incremento.

for i=0, i<=5, i++;

Tabla 3
Ciclo *for*

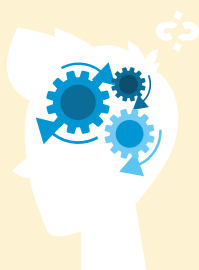
for	Explicación
<pre>num=2; for i=0, for <=5, i++; num=num*i;</pre>	Se inicializa el valor de i en 0, se repite hasta <=5 y se incrementa de uno en uno. num=num*i, toma el valor de i y lo multiplica por el valor de num. Repite 5 veces el procedimiento.

El algoritmo anterior se repite 5 veces dado que el incremento de i es de uno en uno, cada vez que se ejecuta el código.

do-while: haga-mientras. Se ejecuta mientras se cumpla una condición específica dentro del ciclo.

Tabla 4
Do-while

do-while	Explicación
<pre>num=2; i=1; do num=num*i; i++; while (i<=5)</pre>	Se inicializa el valor de num en 2 y de i en 1, Se multiplica $\text{num} * i$ en cada iteración Se incrementa i de uno en uno. Valida que $i \leq 5$ Repite 5 veces el procedimiento.



El algoritmo anterior se repite 5 veces dado que el incremento de i es de uno en uno, cada vez que se ejecuta el código.

Los ciclos son útiles a la hora de realizar tareas repetitivas; su automatización y ejecución eficiente ahorra tiempos y permite minimizar errores.

Diferencia entre secuencia, condiciones y ciclos

Tabla 5
Diferencia entre secuencia, condiciones y ciclos

Concepto	Propósito	Estructura	Ejemplo práctico
Secuencia	Ejecutar instrucciones en orden lógico.	Instrucciones consecutivas.	Multiplicar dos números y presentar el resultado.
Concepto	Según una lógica aplicada se toma una decisión.	<i>if</i> <i>else</i>	Si hace sol, llevar gafas. Si no, llevar sombrilla.
Concepto	Repite una instrucción mientras se cumpla una condición.	<i>while</i> <i>for</i> <i>do-while</i>	Multiplicar un número hasta el 10.

En conclusión, los conceptos de secuencia, condición y ciclo, hacen parte fundamental en el diseño de algoritmos en programación.

La secuencia garantiza un flujo lineal en cada instrucción y las condiciones aportan flexibilidad al código, permitiendo la toma de decisiones y la ejecución de procedimientos, según las respuestas obtenidas; además, los ciclos optimizan los algoritmos a la hora de ejecutar tareas repetitivas de manera mucho más eficiente. Al combinar estos tres elementos, se generan soluciones de alto valor, potente y adaptable a los problemas que se presenten.